

```

import sys

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn import datasets, preprocessing, model_selection, impute
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVR, SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import mean_squared_error, mean_absolute_error,
accuracy_score, confusion_matrix

from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                              QHBoxLayout, QTabWidget, QPushButton, QLabel,
                              QComboBox, QFileDialog, QSpinBox, QDoubleSpinBox,
                              QGroupBox, QScrollArea, QTextEdit, QStatusBar,
                              QProgressBar, QCheckBox, QGridLayout, QMessageBox,
                              QDialog, QLineEdit)

from PyQt6.QtCore import Qt

import tensorflow as tf

from tensorflow.keras import layers, models, optimizers


class EnhancedMLCourseGUI(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Advanced Machine Learning Course GUI")

        self.setGeometry(100, 100, 1600, 900)

```

```

# Initialize main widget and layout
self.main_widget = QWidget()
self.setCentralWidget(self.main_widget)
self.layout = QVBoxLayout(self.main_widget)

# Enhanced data management
self.X_train = None
self.X_test = None
self.y_train = None
self.y_test = None
self.current_model = None

# Create enhanced components
self.create_data_management_section()
self.create_enhanced_tabs()
self.create_advanced_visualization()
self.create_status_bar()

def create_data_management_section(self):
    data_group = QGroupBox("Advanced Data Management")
    data_layout = QGridLayout()

    # Dataset selection with more options
    self.dataset_combo = QComboBox()
    self.dataset_combo.addItems([
        "Load Custom Dataset",
        "Iris Dataset",
        "Breast Cancer Dataset",
        "Boston Housing Dataset",

```

```

        "Digits Dataset"
    ])

    # Missing data handling
    self.imputation_combo = QComboBox()
    self.imputation_combo.addItem([
        "No Imputation",
        "Mean Imputation",
        "Median Imputation",
        "Most Frequent",
        "Forward Fill",
        "Backward Fill"
    ])

    # Scaling options
    self.scaling_combo = QComboBox()
    self.scaling_combo.addItem([
        "No Scaling",
        "Standard Scaling",
        "Min-Max Scaling",
        "Robust Scaling"
    ])

    # Layout arrangement
    data_layout.addWidget(QLabel("Dataset:"), 0, 0)
    data_layout.addWidget(self.dataset_combo, 0, 1)
    data_layout.addWidget(QLabel("Missing Data Strategy:"), 1, 0)
    data_layout.addWidget(self.imputation_combo, 1, 1)
    data_layout.addWidget(QLabel("Scaling Method:"), 2, 0)

```

```
data_layout.addWidget(self.scaling_combo, 2, 1)
```

```
data_group.setLayout(data_layout)
```

```
self.layout.addWidget(data_group)
```

```
def create_enhanced_tabs(self):
```

```
    # Tab system will include more advanced machine learning techniques
```

```
    pass
```

```
def create_advanced_visualization(self):
```

```
    # Advanced visualization with more metrics and plots
```

```
    pass
```

```
def create_status_bar(self):
```

```
    # Status bar with more detailed feedback
```

```
    pass
```

```
def load_dataset(self):
```

```
    # Enhanced dataset loading with more preprocessing options
```

```
    pass
```

```
def apply_imputation(self, X):
```

```
    # Implement various imputation strategies
```

```
    strategy = self.imputation_combo.currentText()
```

```
    if strategy == "No Imputation":
```

```
        return X
```

```
    if strategy == "Mean Imputation":
```

```

        imputer = impute.SimpleImputer(strategy='mean')
    elif strategy == "Median Imputation":
        imputer = impute.SimpleImputer(strategy='median')
    elif strategy == "Most Frequent":
        imputer = impute.SimpleImputer(strategy='most_frequent')

    return imputer.fit_transform(X)

def create_enhanced_svm_tab(self):
    """Create an advanced SVM configuration tab"""
    widget = QWidget()
    layout = QVBoxLayout(widget)

    # SVM Classification Group
    svm_classification_group = QGroupBox("SVM Classification")
    clf_layout = QGridLayout()

    # Kernel Selection
    clf_layout.addWidget(QLabel("Kernel:"), 0, 0)
    self.svm_clf_kernel = QComboBox()
    self.svm_clf_kernel.addItem(["linear", "rbf", "poly", "sigmoid"])
    clf_layout.addWidget(self.svm_clf_kernel, 0, 1)

    # Regularization Parameter C
    clf_layout.addWidget(QLabel("Regularization (C):"), 1, 0)
    self.svm_clf_c = QDoubleSpinBox()
    self.svm_clf_c.setRange(0.001, 10.0)
    self.svm_clf_c.setValue(1.0)
    self.svm_clf_c.setSingleStep(0.1)

```

```
clf_layout.addWidget(self.svm_clf_c, 1, 1)
```

```
# Loss Function for Classification
```

```
clf_layout.addWidget(QLabel("Loss Function:"), 2, 0)
```

```
self.svm_clf_loss = QComboBox()
```

```
self.svm_clf_loss.addItem("hinge", "squared_hinge")
```

```
clf_layout.addWidget(self.svm_clf_loss, 2, 1)
```

```
# Polynomial Degree (for poly kernel)
```

```
clf_layout.addWidget(QLabel("Polynomial Degree:"), 3, 0)
```

```
self.svm_clf_degree = QSpinBox()
```

```
self.svm_clf_degree.setRange(1, 5)
```

```
self.svm_clf_degree.setValue(3)
```

```
clf_layout.addWidget(self.svm_clf_degree, 3, 1)
```

```
svm_classification_group.setLayout(clf_layout)
```

```
layout.addWidget(svm_classification_group)
```

```
# SVM Regression Group
```

```
svm_regression_group = QGroupBox("SVM Regression")
```

```
reg_layout = QGridLayout()
```

```
# Kernel Selection for Regression
```

```
reg_layout.addWidget(QLabel("Kernel:"), 0, 0)
```

```
self.svm_reg_kernel = QComboBox()
```

```
self.svm_reg_kernel.addItem("linear", "rbf", "poly")
```

```
clf_layout.addWidget(self.svm_reg_kernel, 0, 1)
```

```
# Epsilon Parameter
```

```
reg_layout.addWidget(QLabel("Epsilon:"), 1, 0)
self.svm_reg_epsilon = QDoubleSpinBox()
self.svm_reg_epsilon.setRange(0.0, 1.0)
self.svm_reg_epsilon.setValue(0.1)
self.svm_reg_epsilon.setSingleStep(0.01)
reg_layout.addWidget(self.svm_reg_epsilon, 1, 1)
```

```
# Regularization Parameter C for Regression
```

```
reg_layout.addWidget(QLabel("Regularization (C):"), 2, 0)
self.svm_reg_c = QDoubleSpinBox()
self.svm_reg_c.setRange(0.001, 10.0)
self.svm_reg_c.setValue(1.0)
self.svm_reg_c.setSingleStep(0.1)
reg_layout.addWidget(self.svm_reg_c, 2, 1)
```

```
# Loss Function for Regression
```

```
reg_layout.addWidget(QLabel("Loss Function:"), 3, 0)
self.svm_reg_loss = QComboBox()
self.svm_reg_loss.addItem("epsilon_insensitive", "squared_epsilon_insensitive")
reg_layout.addWidget(self.svm_reg_loss, 3, 1)
```

```
svm_regression_group.setLayout(reg_layout)
layout.addWidget(svm_regression_group)
```

```
# Train Button
```

```
train_btn = QPushButton("Train SVM")
train_btn.clicked.connect(self.train_advanced_svm)
layout.addWidget(train_btn)
```

```
return widget
```

```
def train_advanced_svm(self):
```

```
    """Train SVM with advanced configurations"""
```

```
    try:
```

```
        # Check if data is loaded
```

```
        if self.X_train is None or self.y_train is None:
```

```
            self.show_error("Please load a dataset first!")
```

```
            return
```

```
        # Determine problem type based on unique target values
```

```
        is_classification = len(np.unique(self.y_train)) <= 10
```

```
        if is_classification:
```

```
            # SVM Classification
```

```
            model = SVC(
```

```
                C=self.svm_clf_c.value(),
```

```
                kernel=self.svm_clf_kernel.currentText(),
```

```
                degree=self.svm_clf_degree.value() if self.svm_clf_kernel.currentText() ==  
'poly' else 3,
```

```
                loss=self.svm_clf_loss.currentText()
```

```
            )
```

```
            model.fit(self.X_train, self.y_train)
```

```
            y_pred = model.predict(self.X_test)
```

```
        # Calculate classification metrics
```

```
        accuracy = accuracy_score(self.y_test, y_pred)
```

```
        conf_matrix = confusion_matrix(self.y_test, y_pred)
```

```

# Update metrics display
metrics_text = f"SVM Classification Metrics:\n"
metrics_text += f"Accuracy: {accuracy:.4f}\n\n"
metrics_text += "Confusion Matrix:\n"
metrics_text += str(conf_matrix)

else:
    # SVM Regression
    model = SVR(
        C=self.svm_reg_c.value(),
        kernel=self.svm_reg_kernel.currentText(),
        epsilon=self.svm_reg_epsilon.value(),
        loss=self.svm_reg_loss.currentText()
    )
    model.fit(self.X_train, self.y_train)
    y_pred = model.predict(self.X_test)

    # Calculate regression metrics
    mse = mean_squared_error(self.y_test, y_pred)
    mae = mean_absolute_error(self.y_test, y_pred)

    # Update metrics display
    metrics_text = f"SVM Regression Metrics:\n"
    metrics_text += f"Mean Squared Error: {mse:.4f}\n"
    metrics_text += f"Mean Absolute Error: {mae:.4f}"

    # Update visualization and metrics
    self.update_visualization(y_pred)
    self.metrics_text.setText(metrics_text)

```

```
except Exception as e:
```

```
    self.show_error(f"Error training SVM: {str(e)}")
```

```
def main():
```

```
    app = QApplication(sys.argv)
```

```
    window = EnhancedMLCourseGUI()
```

```
    window.show()
```

```
    sys.exit(app.exec())
```

```
if __name__ == '__main__':
```

```
    main()
```